

Indice

1. Introduzione
2. ADT
3. Progettazione
4. Specifica

1. Introduzione

1.1 Panormaica

Il progetto implementa un sistema che gestisca l'utilizzo di un'aula studio universitaria. Esso permette di:

- registrare gli studenti
- registrare le prenotazioni dei posti
- gestire le prenotazioni dei posti
- effettuare check-in
- gestire entrata in aula studio di studenti senza prenotazione

1.2 Com'è strutturato il progetto?

Il progetto è organizzato in moduli, i file Header si trovano nella cartella include, mentre i file C si trovano nella cartella src.

1.3 Assunzioni progettuali

Per questo sistema sono state pensate degli assunti progettuali:

- Capacità aula fissa (80)
- Tre fasce orarie (mattina, primo pomeriggio, secondo pomeriggio)
- Matricola come codice alfanumerico con 8 caratteri

2. ADT

2.1 Analisi del problema

Prima di esplicitare quale ADT è stato utilizzato dobbiamo considerare le condizioni che hanno portato alla scelta che abbiamo fatto.

Il sistema deve gestire quattro categorie principali di informazioni:

- dati studente (nome, cognome, matricola, corso)

I dati di chi è prenotato in una fascia oraria

- I dati di chi non è prenotato ma deve comunque essere inserito in una fascia oraria

In generale tutti questi dati non sono noti a priori e cambiano durante l'esecuzione del programma. Chiaramente in base alle assunzioni fatte precedentemente ci sono dei limiti per alcuni tipi di dato,

Esempio: la capacità dell'aula limita quanti studenti possono essere prenotati per fascia oraria

2.2 Perché lista concatenata

Il progetto utilizza come struttura dati principale una lista concatenata. Comparandola con le altre strutture dati essa è la scelta più sensata in termini di memoria.

Un'alternativa poteva essere un array statico, lo abbiamo scartato per la necessità di una struttura flessibile, in quanto non abbiamo una dimensione fissa degli studenti che si prenotano.

Un'altra alternativa era l'array dinamico, anche questo lo abbiamo scartato per via dei costosi rialloca menti di memoria.

2.3 Operazioni più Utilizzate

Le operazioni di ricerca studente e prenotazione (posto) sono in assoluto quelle più frequentemente utilizzate. Questo perché ogni volta che si effettua una prenotazione, cancellazione o un check-in si invoca una delle due funzioni.

Queste due funzioni condividono lo stesso tipo di complessità computazionale, ovvero una complessità lineare **O(n)** che è dovuta alla natura delle liste, in quanto non ordinate.

Nonostante ciò, è un sacrificio che riteniamo accettabile in quanto il numero di dati è basso.

3. Progettazione

3.1 Architettura dei moduli

Come detto precedentemente il progetto è diviso in cartelle include e src con i rispettivi moduli che comunicano fra loro. Questo per mantenere un certo ordine e riutilizzare il codice quando possibile.

I moduli sono i seguenti:

- main.c
- studente.c/.h
- prenotazione.c/.h
- list.c/.h
- utili.c/.h
- item.c/.h

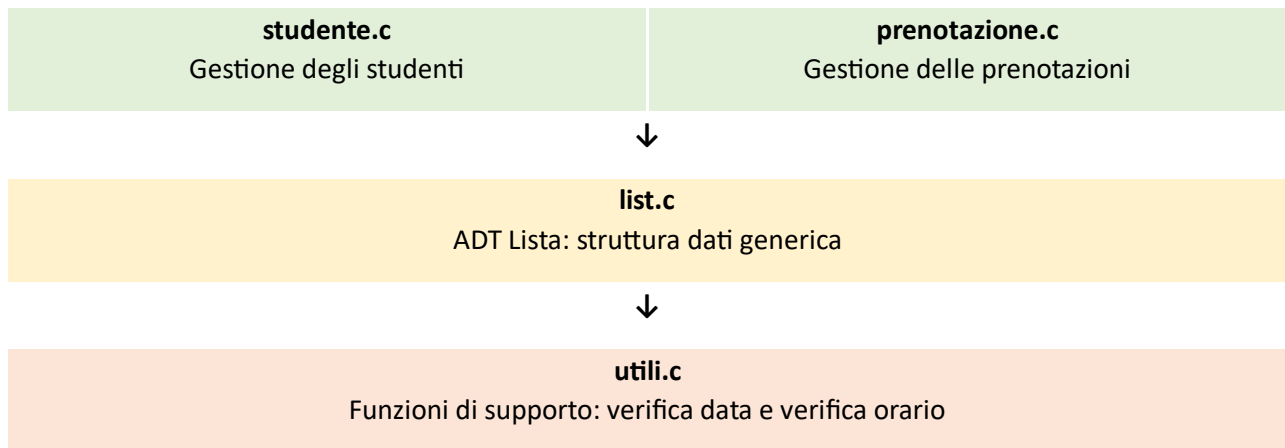
main è l'interfaccia del sistema, list è l'implementazione dell'ADT lista, studente e prenotazione si occupano rispettivamente della gestione dei dati studente e dai dati della prenotazione, utili sono funzioni di supporto (orario e data).

Inoltre, per la lista viene usato il modulo item che indica un tipo generico. Non si trova negli altri moduli.

main.c

Interfaccia utente, menu e controllo del flusso





3.2 Interazione tra i componenti

Studente viene invocato da main per:

- registrare studenti
- cercare studenti
- eliminare studenti

Studente inoltre utilizza la lista per le tre operazioni citate sopra

Prenotazione viene invocato da main per

- creazione prenotazioni
- ricerca prenotazioni
- verifica disponibilità
- check-in
-

Prenotazione usa la lista per:

- inserire prenotazioni
- cercare prenotazioni
- rimuovere prenotazioni
- scorrere le liste orarie

Inoltre Prenotazione fa utilizzo delle funzioni di verifica orario e data che si trovano nel modulo utili.

4. Specifica

4.1 Specifica Sintattica Lista

- Tipo di riferimento: list
- Tipi usati: item, boolean
- Operatori
 - newList() → list
 - emptyList(list) → boolean
 - consList(item, list) → list
 - tailList(list) → list
 - getFirst(list) → item
 - sizeList(list) → integer
 - posItem(list, item) → integer

- `searchItem(list, item) → boolean`
- `reverseList(list) → list`
- `removeItem(list, item) → list`
- `getItem(list, integer) → item`
- `insertList(list, integer, item) → list`
- `removeList(list, integer) → list`
- `outputList(list) → list(void(?))`
- `inputList(int Size) → list`

4.2 Specifica Semantica Lista

Specifica Semantica

- Tipo di riferimento list
 - **list** è l'insieme delle sequenze $L = a_1, a_2, \dots, a_n$ di tipo item
 - L'insieme **list** contiene inoltre un elemento nil che rappresenta la lista vuota (priva di elementi)

Operatori:

- **`newList()` → l**

Post: $l = \text{nil}$

- **`emptyList(l)` → b**

Post: se $l = \text{nil}$ allora $b = \text{true}$ altrimenti $b = \text{false}$

- **`consList(e, l)` → l'**

Post: $l = \langle a_1, a_2, \dots, a_n \rangle$ AND $l' = \langle e, a_1, a_2, \dots, a_n \rangle$

- **`tailList(l)` → l'**

Pre: $l = \langle a_1, a_2, \dots, a_n \rangle$ $n > 0$

Post: $l' = \langle a_2, \dots, a_n \rangle$

- **`getFirst(l)` → e**

Pre: $l = \langle a_1, a_2, \dots, a_n \rangle$ $n > 0$

Post: $e = a_1$

- **`sizeList(l)` → n**

Post: $l = \langle a_1, a_2, \dots, a_n \rangle$ AND $n \geq 0$

- **`searchItem(l, e)` → b**

Post: se e è contenuto in l allora $b = \text{true}$, se no $b = \text{false}$

- **`posItem(l, e)` → p**

Post: se e è contenuto in l allora p è la posizione della prima occorrenza di e in l , altrimenti $p = -1$

- **reverseList(l) → l'**

Post: $l = \langle a_1, a_2, \dots, a_n \rangle$ AND $l' = \langle a_n, \dots, a_2, a_1 \rangle$

- **removeItem(l, e) → l'**

Post: se e è contenuto in l , allora l' si ottiene da l

eliminando la prima occorrenza di e in l , altrimenti $l' = l$

- **getItem(l, pos) → e**

Pre: $pos \geq 0$ AND $sizeList(l) > pos$ // assumiamo 0 come prima posizione

Post: e è l'elemento che occupa in l la posizione pos

- **insertList(l, p, e) → l'**

Pre: $pos \geq 0$ AND $sizeList(l) \geq p$ // assumiamo 0 come prima posizione

Post: l' si ottiene da l inserendo e in posizione p

- **removeList(l, p) → l'**

Pre: $pos \geq 0$ AND $sizeList(l) > p$

Post: l' si ottiene da l eliminando l'elemento in posizione p

4.3 Specifica Sintattica Studente

- Tipo di riferimento: Studente
- Tipi usati: char[] (stringhe), list

Operatori:

- registra_dati_studente → void
- crea_studente → void
- cerca_studente → Studente*
- libera_studente → Studente*

4.4 Specifica Sintattica Studente

Studente S è una tripla che contiene matricola, nome+cognome, corso

Lista studente L è l'insieme delle sequenze S_1, S_2, S_n .

Avviando il programma la lista sarà vuota.

Operatori:

- **registra_dati_studente(s)**

Post: $s.matricola, s.nc, s.corso$ contengono valori validi inseriti dall'utente.

- **crea_studente(s)**

Post: listaStudenti' = <s, listaStudenti>

- **cerca_studente(s, matricola)**

Post: Se esiste uno studente S tale che S.matricola = matricola, allora:

return S

altrimenti:

return NULL

- **libera_studente(s, matricola)**

Se esiste uno studente S con matricola matricola, allora:

listaStudenti' = listaStudenti senza S

altrimenti:

listaStudenti' = listaStudenti

4.5 Specifica Sintattica Prenotazione

- Tipo di riferimento: Prenotazione
- Tipi usati: char[] (stringa), int, list

Operatori:

- crea_posto → Prenotazione*
- registra_prenotazione → void
- cerca_posto → Prenotazione*
- verifica_disponibilità_orario → void
- libera_prenotazione → void
- visualizza_prenotazioni → void
- checkin_studente → void

4.6 Specifica Semantica Prenotazione

Prenotazione P è una tripla che contiene matricola, data, orario

Le liste orario sono LM = lista mattina, LP = lista primo pomeriggio, LS = lista secondo pomeriggio.

Operatori:

- **crea_posto(s, p, matricola)**

Pre: Esiste uno studente S con matricola matricola.

Post: p.matricola = matricola p.data è una data valida p.orario ∈ {1,2,3}

- **registra_prenotazione(s, p)**

Post: Se p.orario = 1 allora LM' = <p, LM> Se p.orario = 2: allora LP' = <p, LP> Se p.orario = 3: allora LS' = <p, LS>

- **cerca_posto(p, matricola)**

Post: Se esiste una prenotazione P con P.matricola = matricola nella lista dell'orario scelto allora return P

altrimenti return NULL

- **verifica_disponibilità_orario(p)**

Post: Conta il numero di prenotazioni con la stessa data e orario. Non modifica le liste.

- **libera_prenotazione(s, matricola, LM, LP, LS)**

Post: Se esiste una prenotazione P con matricola matricola: lista_oraria' = lista_oraria senza P

- **checkin_studente(s, p, LM, LP, LS)**

Post: Se esiste una prenotazione P con matricola matricola, stampa conferma. Non modifica le liste.

Non modifica le liste.

- **visualizza_prenotazioni(LM, LP, LS)**

Post: Stampa tutte le prenotazioni. Non modifica le liste.